

Exercice — Responsive : mobile-first

 **DURÉE** : ● 10 min | ● 20 min | ● 35 min

 **Niveau Bloom** : Appliquer (3) / Créer (6)

 **Prérequis** : Leçon XIII (Responsive), Leçons XI (Flexbox) et XII (Grid) recommandées

Objectif

Écrire un CSS mobile-first avec **media queries** qui adapte un layout à 3 breakpoints (mobile 320px, tablette 768px, desktop 1024px), et **diagnostiquer** un design figé pour le rendre fluide.

● Échauffement

Recopier ce HTML dans `responsive.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="bar">Mon site responsive</header>
  <main class="cards">
    <article class="card">Card 1</article>
    <article class="card">Card 2</article>
    <article class="card">Card 3</article>
    <article class="card">Card 4</article>
  </main>
</body>
</html>
```

Et ce CSS mobile-first dans `style.css` :

```
/* Reset minimal */
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; }

/* ≡ Mobile par défaut (< 768px) ≡ */
.bar {
  background: #2563eb;
  color: white;
  padding: 1rem;
  text-align: center;
}

.cards {
  display: flex;
  flex-direction: column;
  gap: 1rem;
  padding: 1rem;
}

.card {
  background: #e5e7eb;
  padding: 2rem;
  border-radius: 8px;
  text-align: center;
}

/* ≡ Tablette (≥ 768px) ≡ */
@media (min-width: 768px) {
  .cards {
    flex-direction: row;
    flex-wrap: wrap;
  }
  .card {
    flex: 1 1 calc(50% - 0.5rem);
  }
}

/* ≡ Desktop (≥ 1024px) ≡ */
@media (min-width: 1024px) {
  .card {
    flex: 1 1 calc(25% - 0.75rem);
  }
}
```

Ouvrir dans le navigateur. **Tester** :

1. Ouvrir DevTools, activer le mode **device toolbar** (**Ctrl+Shift+M**).
2. Tester les 3 largeurs : 375px (mobile), 800px (tablette), 1200px (desktop).
3. **Observer** : la disposition des cards passe de 1 colonne → 2 colonnes → 4 colonnes.

Pourquoi mobile-first ? La règle de base est définie pour mobile (la plus simple). Les media queries `min-width` ajoutent des règles aux écrans plus larges — pas l'inverse.

● Normal — À vous de jouer

Voici un fichier `pas-responsive.html` que vous devez rendre adaptatif :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Navbar à transformer</title>
</head>
<body>
  <nav class="navbar">
    <a class="navbar-logo" href="#">MaMarque</a>
    <ul class="navbar-menu">
      <li><a href="#">Accueil</a></li>
      <li><a href="#">Produits</a></li>
      <li><a href="#">À propos</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
    <button class="navbar-burger" aria-label="Menu">☰</button>
  </nav>
</body>
</html>
```

CSS de départ (à compléter) :

```
.navbar {
  display: flex;
  /* À COMPLÉTER */
}
.navbar-menu { /* À COMPLÉTER */ }
.navbar-burger { display: none; /* À COMPLÉTER */ }
```

Objectifs :

1. **Desktop (≥ 1024px)** : logo à gauche, menu horizontal à droite, burger caché.
2. **Tablette (768-1023px)** : logo et menu côte à côte mais menu en grille 2 colonnes.
3. **Mobile (< 768px)** : logo en haut, **burger visible**, menu caché par défaut.
4. **Le menu n'a pas besoin d'être interactif** (pas de JS) — juste afficher le burger et **masquer visuellement le menu** (CSS `display: none`).
5. **Mobile-first** strict : règles par défaut = mobile, puis `min-width` pour tablette et desktop.

Bonus : ajouter un breakpoint « XL » ($\geq 1440\text{px}$) qui passe le `font-size` du logo à `1.5rem`.

Livrable : `pas-responsive.html` complété + CSS mobile-first commenté.

● Défi — Container queries

Les **container queries** (`@container`) sont la nouvelle façon de faire du responsive : au lieu de réagir à la **largeur de la viewport**, on réagit à la **largeur du conteneur parent** d'un élément.

Lire : **MDN — Container queries**.

Exercice : créer un composant **Card** qui s'adapte à son conteneur.

Maquette HTML :

```
<div class="sidebar">
  <article class="card">
    
    <h3>Titre</h3>
    <p>Texte court</p>
  </article>
</div>

<div class="hero">
  <article class="card">
    
    <h3>Titre</h3>
    <p>Texte court</p>
  </article>
</div>
```

Cible :

- Si la card est dans un conteneur $< 400\text{px}$ (sidebar) : layout vertical (image au-dessus, texte dessous).
- Si la card est dans un conteneur $\geq 400\text{px}$ (hero pleine largeur) : layout horizontal (image à gauche, texte à droite).
- **Le composant `.card` ne sait pas dans quel contexte il vit** — il s'adapte tout seul.

CSS à compléter :

```
.sidebar { width: 300px; container-type: inline-size; }  
.hero    { width: 100%; container-type: inline-size; }  
  
.card {  
  /* layout vertical par défaut */  
}  
  
@container (min-width: 400px) {  
  .card {  
    /* layout horizontal */  
  }  
}
```

Questions :

1. Quelle est la **différence pratique** entre `@media (min-width: 400px)` et `@container (min-width: 400px)` ?
2. Quel est le support navigateur des container queries en 2026 ? Acceptable en prod ?
3. **Quel cas d'usage justifie** vraiment les container queries vs les media queries classiques ?
(Indice : penser à une grille de cards où certaines sont en sidebar et d'autres en pleine page **dans la même page**.)

Livrable : un fichier `container-queries.html` fonctionnel avec démo + réponses aux 3 questions.